

COALGUARD

Final Code Verification, Bug Fixing, Integration Testing and Hackathon Readiness

One-shot Gemini prompt for auditing the implemented CoalGuard Governance Platform, fixing real defects, validating end-to-end behavior, and preparing the project for a live hackathon demonstration.

Usage: Attach this PDF or paste its contents into Gemini alongside the CoalGuard project workspace. Instruct Gemini to inspect and execute the actual codebase, not merely review the implementation summary.

Coverage: startup, database, seed/reset, authentication, RBAC, inspection, evidence security, offline PWA, corrective actions, audit trail, compliance, dashboard, risk index, recurrence, anomaly detection, GIS, notifications, OCR, API contracts, frontend errors, PWA, responsiveness, security, code quality, demo data, full end-to-end testing, fixes, documentation, demo readiness, judge defense, and final acceptance.

GEMINI PROMPT: FINAL CODE VERIFICATION, BUG FIXING, INTEGRATION TESTING AND HACKATHON READINESS FOR COALGUARD

You are the **senior QA engineer, backend engineer, frontend engineer, security reviewer, and hackathon technical lead** responsible for taking the existing CoalGuard Governance Platform from "implemented" to **verified, stable, demo-ready, and technically defensible**.

The project has already been implemented according to a 48-hour plan.

You **MUST** inspect the actual codebase and execute the application where possible.

Do **NOT** trust implementation summaries or claims that a feature is complete.

Your job is to verify the real system.

PROJECT

CoalGuard Governance Platform

The platform is an AI-enabled governance and compliance monitoring system for coal mining operations.

Core workflow:

Field Observation -> Evidence -> Violation -> Corrective Action -> Assignment -> Action -> Verification -> Closure -> Audit Trail -> Risk/Dashboard Update

The platform currently contains approximately:

Backend

- FastAPI
- PostgreSQL
- JWT authentication
- RBAC
- Compliance APIs
- Inspection APIs
- Corrective-action APIs
- Dashboard APIs
- Governance/risk logic
- Audit logging
- Deterministic seed/reset functionality

Frontend

- React
- Vite

- PWA
- IndexedDB
- React Router
- Recharts
- Leaflet / React-Leaflet
- Axios
- Lucide icons
- Vanilla CSS

Potential files/modules already created include:

```

backend/
  main.py
  services/
    governance.py
    audit.py
    inspections.py
    actions.py
    compliance.py
    dashboard.py

frontend/
  src/
    lib/
      api.js
      idb.js
    pages/
      Login.jsx
      CorporateDashboard.jsx
      MineDashboard.jsx
      Inspect.jsx
      Map.jsx
    components/
      AccountabilityTimeline.jsx
  App.jsx
  main.jsx
  index.css

```

There is also a `walkthrough.md` describing the local setup and demo flow.

ABSOLUTE RULES

1. Inspect reality, not claims

Do not assume:

"This feature was implemented"

means:

"This feature actually works."

Check the source code, database schema, routes, frontend calls, and execution behavior.

2. Do not rewrite the project unnecessarily

Preserve the existing architecture unless there is a genuine correctness or integration problem.

Do not replace the stack simply because you personally prefer another framework.

3. Prioritize working functionality

Do not spend time adding visual polish while core workflows are broken.

4. Never fabricate test results

If something cannot be executed, explicitly state that it could not be verified.

5. Fix actual bugs

Do not just report them.

For important issues, modify the code and retest.

6. Avoid scope creep

Do NOT add:

- blockchain
- Kubernetes
- microservices
- native mobile app
- unnecessary LLM chatbot
- unnecessary IoT layer
- deep learning without data

The goal is a reliable hackathon system.

7. No fake AI claims

The governance risk score may be a configurable weighted index.

Do not describe it as validated predictive ML unless the implementation and data genuinely support it.

8. Never use em dashes in documentation or generated text

Use commas, parentheses, colons, or normal hyphens.

PART 1: FIRST, AUDIT THE ENTIRE CODEBASE

Before making changes, inspect:

```
backend/  
frontend/  
database/  
seed files  
docker files  
requirements  
package.json  
configuration
```

README
walkthrough.md
task.md

Create an internal understanding of:

- backend entry point
- route registration
- database connection
- models/schema
- seed logic
- auth mechanism
- RBAC
- audit implementation
- risk implementation
- frontend routes
- API client
- IndexedDB implementation
- PWA configuration
- GIS implementation
- dashboard data flow

Check whether the documentation matches the actual implementation.

PART 2: STARTUP VERIFICATION

Actually attempt to start the system.

Verify:

```
PostgreSQL
↓
Backend
↓
Frontend
```

Test the intended startup sequence.

Check for:

- missing dependencies
- Python import failures
- npm dependency issues
- environment variable problems
- database connection failures
- migrations/schema failures
- CORS problems

- frontend API URL mismatch
- port conflicts
- PWA build errors

If something fails:

1. identify the root cause
2. fix it
3. rerun
4. document what changed

PART 3: DATABASE VERIFICATION

Inspect the actual schema.

Verify:

- tables exist
- PKs are correct
- FKs are correct
- constraints exist
- indexes exist where required
- enum/state values are consistent
- dates and timestamps are correct
- nullable fields are appropriate

Specifically inspect:

```
users
roles
mines
compliance_records
inspections
observations
violations
corrective_actions
audit_logs
risk_scores
notifications
documents
```

Use the actual schema rather than assuming these exact names exist.

Identify any mismatch between:

- backend models
- database schema
- frontend expectations
- seed data

Fix inconsistencies.

PART 4: SEED DATA AND RESET TEST

Run the actual seed/reset mechanism.

Verify:

```
Reset
↓
Seed
↓
Known data state
```

Test reset more than once.

The result should be deterministic.

Check:

- record counts
- expected mines
- expected violations
- expected corrective actions
- expected risk ranking
- expected overdue actions
- expected audit history

The system must recover from a demo mistake without manual SQL edits.

If deterministic IDs are used, verify that they remain stable across resets.

PART 5: AUTHENTICATION TESTING

Test actual login requests.

Verify:

Inspector

- login works
- token generated
- protected endpoint access works
- manager-only endpoints return 403

Mine Manager

- login works
- mine-level access works
- corporate-only views are restricted

Corporate Manager

- login works
- all mines are visible
- enterprise dashboard works

Verify:

- invalid credentials
- missing token
- expired token
- malformed token
- unauthorized role

Fix all critical auth failures.

PART 6: RBAC SECURITY AUDIT

Do NOT trust frontend hiding alone.

Verify that authorization is enforced on the backend.

Test:

```
Inspector → manager endpoint
Inspector → verify corrective action
Inspector → close action
Mine Manager → another mine's restricted resource
```

Expected result:

```
401 or 403
after considering the actual authentication/authorization state
```

Check for IDOR vulnerabilities such as:

```
/inspections/{id}
/corrective-actions/{id}
/mines/{id}
```

Make sure users cannot simply replace an ID and access unauthorized records.

PART 7: FIELD INSPECTION TEST

Test the actual inspection workflow.

Required flow:

```
Login as Inspector
↓
Open inspection form
↓
Capture GPS
↓
Enter observation
↓
Set severity
↓
```

Attach photo
↓
Submit
↓
Backend record created

Test validation cases.

Case 1

High severity + no photo.

Expected:

422
Photo required

Case 2

Missing GPS.

Expected:

422
GPS required

Case 3

Valid inspection.

Expected:

201 / success

Then verify the record exists in PostgreSQL.

Also check:

- timestamp
- latitude
- longitude
- severity
- inspector ID
- mine ID
- photo reference

PART 8: PHOTO AND EVIDENCE SECURITY

Inspect file upload behavior.

Verify:

- allowed MIME types

- maximum file size
- filename handling
- storage path handling
- duplicate behavior
- invalid file rejection

Prevent obvious path traversal such as:

```
../../../../something
```

If hashes are implemented, verify they actually correspond to the uploaded bytes.

Do not claim cryptographic integrity unless it is actually implemented.

PART 9: OFFLINE PWA TEST

This is one of the most important features.

Actually test the application with network disabled.

Expected flow:

```

Open PWA
↓
Network OFF
↓
Create inspection
↓
Save locally
↓
Pending = 1
↓
Network ON
↓
Automatic/manual sync
↓
Server record created
↓
Pending = 0

```

Inspect IndexedDB.

Verify:

- temporary UUID
- record data
- sync status
- retry behavior

Important:

Inspections are create-only for MVP.

Do NOT implement complex conflict resolution unless the existing architecture truly requires it.

Use idempotency to prevent duplicates.

Test:

Sync same inspection twice

Expected:

Exactly one server record

Fix any duplicate creation problems.

PART 10: CORRECTIVE ACTION STATE MACHINE

This is the core governance workflow.

Verify actual state transitions.

Expected:

```
Open
↓
Assigned
↓
In Progress
↓
Verification
↓
Closed
```

Optional:

```
Verification
↓
Reopened
```

For each transition, verify:

- who can trigger it
- required fields
- evidence requirement
- audit event
- timestamp

Especially test:

Action Owner → Verify Own Action

Expected:

403

Then:

Different authorized manager → Verify

Expected:

Success
Status = Closed
Audit event created

If verification failure exists:

Verification → Reopened

must behave correctly.

PART 11: AUDIT TRAIL VERIFICATION

There was an architectural decision to keep auditability as a strong governance feature.

Inspect how audit logging is actually implemented.

The implementation summary claims:

immutable Audit Log trigger system

But the recommended hackathon architecture previously favored application-level audit inserts.

Determine what the code ACTUALLY uses.

Do not assume.

Then verify:

- actor ID
- entity type
- entity ID
- timestamp
- action
- before state
- after state

Check that important state transitions create audit events.

Verify users cannot modify or delete audit entries through normal application APIs.

If audit behavior differs from documentation, update the documentation to match reality.

Do NOT add blockchain.

PART 12: ACCOUNTABILITY TIMELINE

Open a real corrective action.

Verify the timeline displays events such as:

CREATED
ASSIGNED
IN_PROGRESS
ESCALATED
EVIDENCE_UPLOADED
VERIFIED

CLOSED

where applicable.

Check that:

- ordering is correct
- actors are visible
- timestamps are correct
- state transitions are understandable
- missing events do not crash the component

If the timeline depends on missing audit events, fix instrumentation rather than hiding the problem.

PART 13: COMPLIANCE ENGINE

Test:

- compliance record creation
- due date
- responsible person
- status
- overdue detection
- dashboard aggregation

Create test cases:

Due in future

Status should not be overdue.

Due today

Verify intended behavior.

Due date in past

Must become overdue.

Check timezone handling.

Avoid inconsistent treatment of local date versus UTC timestamp.

PART 14: DASHBOARD VERIFICATION

Verify that dashboard numbers are actually derived from backend data.

Check:

- total mines
- compliance rate
- open violations
- overdue actions
- critical/high-risk mines
- risk ranking
- trend
- aging chart

Perform a manual database calculation for several values and compare it with UI.

Example:

Database:
12 overdue actions

UI:
12 overdue actions

If there is a mismatch, find the root cause.

Do not hardcode dashboard metrics.

PART 15: GOVERNANCE RISK INDEX

Inspect the actual risk implementation.

Confirm whether it is:

- weighted index
- rule engine
- statistical model
- ML model

Do not call it ML unless the code truly implements ML.

Verify:

- input normalization
- weights
- score range
- thresholds
- ranking
- top reasons

A score such as:

82 / 100

must have understandable reasons.

Example:

```
3 high-severity findings remain open
5 overdue actions
Compliance rate = 45%
2 recurring violations
```

Verify that the reasons correspond to actual seeded data.

PART 16: RISK MANIPULATION TEST

Test whether changing an input actually changes the score.

For example:

```
Increase overdue actions
↓
Risk should increase
```

and:

```
Resolve several violations
↓
Risk should improve
```

Do not allow the dashboard to show static fake risk values if the backend is expected to calculate them dynamically.

If risk scores are precomputed for demo purposes, document that clearly.

PART 17: RECURRING VIOLATION LOGIC

If recurrence detection exists, verify the actual algorithm.

For example:

```
category + zone + contractor
```

within a defined time window.

Create test data with:

```
same category
same area
same contractor
3 occurrences
```

Expected:

```
Recurring = true
```

Then create:

```
same category
different contractor
```

and verify expected behavior.

Do not claim sophisticated pattern recognition if the implementation is a SQL/grouping rule.

PART 18: PRODUCTION ANOMALY LOGIC

If production anomaly detection exists, inspect it.

Prefer a transparent method such as:

```
rolling mean
rolling standard deviation
z-score
```

Verify:

```
normal values → no anomaly
large deviation → anomaly
```

Check behavior around:

- missing values
- insufficient history
- zero variance

Do not claim predictive forecasting if the system only detects deviations.

PART 19: GIS TEST

Verify:

```
Inspection GPS
↓
Database
↓
GIS API
↓
Leaflet
↓
Marker
```

The marker must correspond to the actual coordinates.

Clicking a marker should show useful evidence such as:

- mine
- date
- inspector
- severity
- observation
- photo

Test severity filtering.

Do not introduce complex geographic polygons unless the existing data supports them.

PART 20: NOTIFICATION AND ESCALATION TEST

Inspect actual implementation.

Test:

```
Deadline approaching
Overdue
Overdue > escalation threshold
Critical observation
```

Verify notification records.

If no external email service exists, do not fake an email being sent.

An in-app notification record is acceptable for the prototype.

Document the actual behavior.

PART 21: OCR TEST

Only test this if implemented.

Verify:

```
Upload document
↓
OCR
↓
Text
↓
Structured fields
↓
Human review
↓
Draft compliance/observation record
```

Most importantly:

OCR must connect to the governance workflow.

If the OCR feature only displays extracted text and has no downstream purpose, classify it as optional rather than core.

Do not waste time perfecting OCR if P0 functionality is unstable.

PART 22: API CONTRACT AUDIT

Compare:

```
Backend routes
vs
Frontend API calls
```

Identify:

- missing endpoint
- wrong HTTP method
- wrong parameter name

- wrong response field
- wrong content type
- authentication mismatch

Test all critical endpoints.

Check error responses.

Preferred:

```
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict
422 Validation Error
500 Server Error
```

Do not expose internal stack traces to frontend users.

PART 23: FRONTEND ERROR HANDLING

Test:

- backend unavailable
- timeout
- 401
- 403
- 404
- 422
- file upload failure
- sync failure

The application should show a useful UI message instead of:

```
Cannot read properties of undefined
```

or a blank screen.

Add sensible loading states and empty states where missing.

PART 24: PWA VERIFICATION

Check:

- manifest
- service worker
- installability
- caching
- offline fallback

- IndexedDB
- reconnect behavior

Run:

```
npm run build
```

and confirm the production build succeeds.

Do not claim the application is offline-capable simply because a service worker file exists.

Actually verify offline behavior.

PART 25: RESPONSIVE DESIGN

Test at:

- desktop
- laptop
- tablet
- phone

The inspector workflow should work comfortably on a phone.

The corporate dashboard should remain usable on desktop.

Do not spend hours on animation.

Prioritize:

- readable typography
- clear status indicators
- usable buttons
- visible error states
- no horizontal overflow

PART 26: SECURITY REVIEW

Perform a practical security audit.

Check:

- hardcoded secrets
- database password exposure
- JWT secret exposure
- API keys in frontend
- unrestricted CORS
- SQL injection

- unsafe file uploads
- IDOR
- insecure admin endpoint
- unrestricted seed/reset endpoint
- password storage
- token storage

Important:

A public admin reset endpoint is dangerous in production.

For hackathon use, either:

- protect it with admin authorization
- disable it outside demo mode
- or keep it as a local-only command

Prefer the safest practical solution.

PART 27: CODE QUALITY

Review:

- duplicated logic
- unused dependencies
- dead code
- missing exception handling
- inconsistent naming
- giant React components
- duplicated API logic
- hardcoded IDs
- hidden magic numbers
- brittle URLs

Do not refactor everything.

Only fix issues that materially affect:

- correctness
- maintainability
- demo stability
- security

PART 28: DEMO DATA VALIDATION

Verify that the seed dataset creates a compelling story.

The demo should contain:

Mine A

Low risk

Mine B

Medium risk

Mine C

Critical risk

Critical mine should contain:

- several overdue actions
- high severity findings
- at least one repeated violation
- realistic audit timeline
- enough data to explain high risk

Do not create obviously artificial nonsense data.

Names, timestamps, categories, and numbers should appear plausible.

PART 29: FULL END-TO-END TEST

This is the MOST IMPORTANT TEST.

Run the complete demo from a fresh reset.

Step 1

Reset database.

Step 2

Login as Corporate Manager.

Step 3

Show enterprise dashboard.

Step 4

Open the highest-risk mine.

Step 5

Inspect risk reasons.

Step 6

Switch to Inspector.

Step 7

Turn network OFF.

Step 8

Create a high-severity inspection.

Step 9

Capture GPS.

Step 10

Attach evidence.

Step 11

Save offline.

Step 12

Turn network ON.

Step 13

Verify synchronization.

Step 14

Create or confirm corrective action.

Step 15

Assign owner.

Step 16

Move action through:

Assigned
→ In Progress
→ Verification

Step 17

Attempt self-verification.

Expected:

403

Step 18

Login as another authorized manager.

Step 19

Verify and close.

Step 20

Open accountability timeline.

Step 21

Return to dashboard.

Step 22

Confirm the new state is visible.

Step 23

Open GIS map.

Step 24

Confirm observation appears geographically.

The complete flow must work without manual database editing.

PART 30: DEMO STABILITY

Create a pre-demo checklist.

Verify:

- Docker starts
- backend starts
- frontend starts
- database seeded
- credentials known
- browser tabs prepared
- mobile/PWA ready

- map works
- dashboard works
- reset command works
- backup demo path exists

Create:

```
DEMO_RESET  
DEMO_CREDENTIALS  
DEMO_DATA_CHECK
```

where appropriate.

PART 31: FINAL PRIORITIZATION

After testing, classify all identified issues.

P0 Blocking

Anything that breaks:

- startup
- login
- database
- inspection
- corrective action
- dashboard
- risk
- audit
- offline flow

P1 Important

Examples:

- GIS bug
- notification issue
- timeline UI issue
- recurrence logic
- responsive bug

P2 Polish

Examples:

- animation
- styling

- minor typography
- optional OCR improvement

Fix P0 first.

Then P1.

Do not spend time on P2 while P0 remains broken.

PART 32: IMPLEMENT THE FIXES

For every P0 bug:

1. Explain root cause.
2. Modify the code.
3. Run the relevant test.
4. Run regression test.
5. Confirm the fix.

Do the same for high-impact P1 issues.

Do NOT simply provide recommendations.

Actually edit files where possible.

PART 33: UPDATE DOCUMENTATION

After fixing the code, update:

```
walkthrough.md  
README.md  
task.md
```

so documentation matches reality.

The documentation must accurately state:

- actual startup process
- actual architecture
- actual audit mechanism
- actual risk implementation
- actual offline mechanism
- actual roles
- actual APIs
- actual demo workflow

Never document a feature that does not actually work.

PART 34: FINAL SYSTEM REPORT

At the end, produce a clear report with:

1. VERIFIED FEATURES

Feature | Status | Evidence

2. FIXES MADE

Bug | Root Cause | Fix | Verification

3. REMAINING ISSUES

Issue | Priority | Impact | Recommendation

4. ACTUAL ARCHITECTURE

Show what the code truly implements.

5. ACTUAL AI/ANALYTICS

Explain exactly what is algorithmic, rule-based, statistical, or ML.

6. SECURITY FINDINGS

List important findings and fixes.

7. DEMO READINESS

Give:

READY
or
NOT READY

with reasons.

8. FINAL DEMO SEQUENCE

Give the exact 7-minute flow.

PART 35: FINAL 7-MINUTE DEMO

Optimize the presentation around this story:

0:00 to 1:00

Corporate dashboard

Show:

- mine ranking
- risk
- compliance
- overdue actions

1:00 to 2:00

Explain why a mine is risky

Show:

- risk score
- top factors
- recurring issue
- overdue aging

2:00 to 4:00

Field inspection

Use mobile/PWA:

- turn network off
- capture GPS
- attach evidence
- save offline
- reconnect
- sync

4:00 to 5:30

Corrective action

Show:

- finding
- owner
- deadline
- progress
- verification
- closure

5:30 to 6:15

Accountability

Show:

- timeline

- actors
- timestamps
- escalation

6:15 to 7:00

GIS + final dashboard

Show:

- location
- issue
- evidence
- updated status

Do not show every feature.

Focus on the strongest governance story.

PART 36: FINAL JUDGE DEFENSE

Prepare technically honest answers for these questions:

"Where is the AI?"

Answer based on the ACTUAL implementation.

"Do you have real mining data?"

Explain synthetic data honestly.

"Is your risk score really predictive?"

Do not claim it is predictive unless validated.

"Why not blockchain?"

Explain why an append-only audit trail is sufficient for the prototype.

"Why PWA instead of native mobile?"

Explain development speed and offline capability.

"How does offline synchronization work?"

Explain IndexedDB, temporary UUID, queue, and idempotent sync.

"What happens if someone changes a violation?"

Explain audit trail and state transition logging.

"Can this scale to hundreds of mines?"

Explain current modular architecture and production evolution without claiming that the hackathon prototype is production-ready.

"What makes this different from an Excel dashboard?"

Explain:

field evidence + workflow + accountability + verification + audit trail + explainable risk

FINAL ACCEPTANCE GATE

Do NOT declare the project complete until this minimum checklist passes:

- ☐ Backend starts
- ☐ Frontend starts
- ☐ Database initializes
- ☐ Seed reset works
- ☐ Inspector login works
- ☐ Mine Manager login works
- ☐ Corporate Manager login works
- ☐ RBAC enforced server-side
- ☐ Inspection creation works
- ☐ GPS validation works
- ☐ High severity photo validation works
- ☐ Photo upload works
- ☐ Offline inspection works
- ☐ Offline sync works
- ☐ Duplicate sync prevented
- ☐ Compliance overdue logic works
- ☐ Corrective action state machine works
- ☐ Owner cannot self-verify
- ☐ Valid verification works
- ☐ Audit events are written
- ☐ Timeline renders correctly
- ☐ Dashboard values match database
- ☐ Risk score is explainable
- ☐ Risk ranking works
- ☐ Recurrence logic works, if implemented
- ☐ GIS marker works
- ☐ Notifications work as actually implemented
- ☐ API errors handled
- ☐ Frontend does not crash on backend errors
- ☐ Production frontend build works
- ☐ Full demo works from clean reset
- ☐ Documentation matches implementation

FINAL INSTRUCTION

Treat this as a **real software release candidate review**.

Do not praise the implementation merely because many files exist.

Do not generate a theoretical audit.

Inspect the actual project, run it, test it, find failures, fix them, and verify the fixes.

The desired final state is:

A small, reliable, technically credible CoalGuard prototype that can survive a live 7-minute hackathon demonstration without manual database intervention.

Do not add features merely to increase feature count.

Do not use vague statements.

Do not use unsupported claims.

Do not use em dashes anywhere.